

Nesne Tabanlı Programlamaya Giriş

NTP II - Hafta 1



Hoş Geldiniz!

Bu ders, Python programlama dilini kullanarak Nesne Tabanlı Programlama (OOP) kavramlarını anlamana ve uygulamanıza yönelik 14 haftalık bir yolculuğun başlangıcıdır. İlk hafta, Python'ın temel sözdizimi, veri tipleri ve programlama felsefesi üzerine odaklanarak güçlü bir temel oluşturacağız.

Hafta 1'in temel hedefleri şunlardır:

- Python'ın okunabilir sözdizimi ve felsefesini hatırlamak.
- Değişken tanımlama, PEP 8 kuralları ve temel veri tiplerini (integer, float, string, boolean) doğru kullanabilmek.
- `type()` fonksiyonu ile veri tiplerini kontrol etmek ve `input()` ile kullanıcıdan veri almak.
- `int()`, `str()`, `float()` gibi fonksiyonlarla veri tipleri arasında dönüşümler yapabilmek.

Dersin Genel Yapısı ve Hedefleri

Dersin başında, bu kursun genel yapısını, belirlenen hedeflerini, öğrencilerden beklenen proje çalışmalarını ve başarıyı değerlendirme kriterlerini net bir şekilde açıklamak önemlidir. Bu sayede, tüm öğrenciler dersin gerekliliklerini ve kazanımlarını başından itibaren anlayacaktır.

Bu kursun genel hedefleri şunlardır:

- Python dilinin Nesne Tabanlı Programlama prensiplerini anlamak.
- Gerçek dünya problemlerini çözmek için OOP yaklaşımlarını kullanabilme becerisi geliştirmek.
- Python'ın zengin kütüphane ekosistemini etkin bir şekilde kullanabilmek.

Proje Beklentileri ve Değerlendirme Kriterleri

Bu derste pratik uygulamalar büyük önem taşımaktadır. Dönem boyunca çeşitli projeler geliştirmeniz beklenecektir. Bu projeler, öğrendiğiniz teorik bilgileri gerçek dünya senaryolarına uygulamanıza olanak tanıyacak ve problem çözme becerilerinizi pekiştirecektir.

Değerlendirme kriterleri genellikle şunları kapsar:

- Proje teslim tarihine uyum.
- Kodun doğruluğu ve işlevselliği.
- Kodun okunabilirliği ve PEP 8 standartlarına uygunluğu.
- Kullanılan OOP prensiplerinin etkinliği ve doğruluğu.
- Çözümün özgünlüğü ve karmaşıklığı.

Programlamaya Yeniden Merhaba ve Python Ekosistemi

Bu haftanın ana konusu, Python programlama dilinin temellerine giriş yapmak ve ekosistemini anlamaktır. Programlamaya yeni başlayanlar veya farklı dillerden gelenler için Python'a hızlı ve etkili bir başlangıç sağlamayı hedefliyoruz.

Ana konular:

- Python'ın temel sözdizimi ve veri tipleri.
- `type()` ve `input()` fonksiyonlarının kullanımı.
- Veri tipi dönüşümleri (`int()`, `str()`, `float()`).
- Değişken kavramı ve dinamik tiplendirme.

Temel Yetkinlikleri

Bu hafta sonunda sahip olmanız beklenen temel yetkinlikler şunlardır:

- Temel sözdizimi ve veri tipleri: Python'ın temel yazım kurallarını ve yaygın olarak kullanılan veri tiplerini tanımak ve doğru bir şekilde kullanabilmek.
- PEP 8 isimlendirme kuralları: Python kodunda değişken, fonksiyon ve sınıf adlandırmaları için belirlenen standartları anlamak ve uygulamak.
- Değişken tanımlama ve kullanımı: Programlarda verileri depolamak için değişkenleri tanımlayabilmek ve bunlara değer atayabilmek.
- `type()` ve `input()` fonksiyonları: Bir değişkenin veri tipini dinamik olarak kontrol etmek ve kullanıcıdan veri alabilmek için bu fonksiyonları etkin kullanmak.
- Veri tipi dönüşümleri: Gerekli durumlarda veri tipleri arasında bilinçli dönüşümler yapabilmek.

Python'ın Popülerliği ve Avantajları

Python, günümüzde backend geliştirme, veri bilimi ve otomasyon gibi birçok alanda en popüler programlama dillerinden biridir. Bu popülerliğin arkasında yatan temel avantajları kavramak, dilin gücünü anlamak için önemlidir.

Python'ın temel avantajları:

- **Basit Sözdizimi:** Öğrenmesi kolay, açık ve anlaşılır bir sözdizimine sahiptir. Bu, özellikle yeni başlayanlar için büyük bir avantajdır.
- **Zengin Kütüphane Ekosistemi:** Çok sayıda hazır kütüphane ve çerçeveye sahip olması, geliştiricilerin birçok farklı alanda hızlı ve verimli çözümler üretmesini sağlar.
- **Çok Yönlülük:** Web geliştirmeden yapay zekaya, bilimsel hesaplamalardan ağ programlamaya kadar geniş bir uygulama alanına sahiptir.
- **Platform Bağımsızlığı:** Çoğu işletim sisteminde (Windows, macOS, Linux) çalışabilir.



Python Felsefesi: Okunabilirlik ve Açıklık

Python'ın felsefesi, kodun okunabilirliğine ve açıklığına büyük önem verir. Bu, "Zen of Python" olarak bilinen ilkelerle ifade edilir. Kodun okunabilir olması, hem bireysel geliştiriciler hem de ekip çalışmaları için sürdürülebilirliği ve hatasızlığı artırır.

Zen of Python'dan bazı ilkeler (PEP 20):

- Beautiful is better than ugly. (Güzel, çirkinden iyidir.)
- Explicit is better than implicit. (Açık olan, örtük olandan iyidir.)
- Simple is better than complex. (Basit olan, karmaşıktan iyidir.)
- Complex is better than complicated. (Karmaşık olan, komplike olandan iyidir.)
- Readability counts. (Okunabilirlik önemlidir.)

Bu ilkeler, Python kodunu yazarken her zaman göz önünde bulundurulmalıdır.

Basit Sözdizimi ve Okunabilirlik

Python'ın başarısının temel nedenlerinden biri, basit ve okunabilir sözdizimidir. Bu, karmaşık algoritmaların bile kısa ve anlaşılır kodlarla ifade edilebilmesini sağlar.

Örnek:

Diğer dillerde parantez veya noktalı virgül gerektirebilirken

Python'da daha doğal bir dil yapısı kullanılır.

```
sayi1 = 10
```

```
sayi2 = 20
```

```
toplam = sayi1 + sayi2
```

```
print("Toplam:", toplam)
```

Python, kod bloklarını belirtmek için girintileme (indentation) kullanır, bu da kodun görsel olarak daha düzenli ve okunabilir olmasını sağlar.



Python'ın Zengin Kütüphane Ekosistemi

Python, hem standart kütüphanesi hem de üçüncü taraf paketleriyle inanılmaz derecede zengin bir kütüphane ekosistemine sahiptir. Bu, geliştiricilerin tekerleği yeniden icat etmek zorunda kalmadan birçok farklı görevi yerine getirmesine olanak tanır.

Örnek kütüphane alanları:

- Web Geliştirme: Django, Flask
- Veri Bilimi ve Analizi: NumPy, Pandas, Matplotlib, Scikit-learn
- Yapay Zeka ve Makine Öğrenimi: TensorFlow, PyTorch, Keras
- Otomasyon: Selenium, Paramiko
- Oyun Geliştirme: Pygame

Bu kütüphaneler, Python'ın çeşitli uygulama alanlarında popülerliğini artıran önemli faktörlerdir.

Değişken Kavramı: Geleneksel Tanım

Geleneksel olarak, bir değişken, bir programlama dilinde bir değeri depolamak için kullanılan isimlendirilmiş bir "kutucuk" veya "bellek alanı" olarak düşünülebilir. Bu kutucuğun içindeki değer zamanla değişebilir.

Örnek:

- `yaş = 30` (yaş değişkeni 30 değerini saklar)
- `isim = "Ayşe"` (isim değişkeni "Ayşe" değerini saklar)

Bu tanım, programlamaya yeni başlayanlar için anlaşılması kolaydır. Ancak Python'daki değişkenlerin çalışma mantığını tam olarak açıklamaz.

Python'da Değişkenler: Referanslar ve Etiketler

Python'da değişkenler sadece veri saklayan kutular değildir. Daha doğru bir ifadeyle, değişkenler bellekteki nesnelere işaret eden birer etiket veya referanstır. Bir değer atadığımızda, aslında bir nesne oluşturulur ve değişken bu nesneyi işaret eder.

`x = 10` `# 10 değeri bellekte bir 'int' nesnesi olarak oluşur.`

`# x değişkeni bu 'int' nesnesine referans verir.`

`y = x` `# y de aynı 10 nesnesine referans veri`

Bellekteki Nesneler ve Referanslar

Python'da her şey bir nesnedir. Bir değişkene bir değer atadığınızda, o değer bellekte bir nesne olarak oluşturulur. Değişken adı ise bu nesnenin bellek adresine işaret eden bir referanstır.

Aynı nesneyi işaret eden birden fazla değişken:

```
a = [1-3] # Bir liste nesnesi oluşturulur, 'a' onu işaret eder.
```

```
b = a      # 'b' de aynı liste nesnesini işaret eder.
```

```
b.append(4)
```

```
print(a)    # Çıktı: [1-4]
```

```
print(b)    # Çıktı: [1-4]
```

Burada a ve b aynı liste nesnesine referans verdiği için b üzerinden yapılan değişiklik a'yı da etkiler.

OOP İçin Değişken Kavramının Önemi

Değişkenlerin birer etiket veya referans olduğu temel anlayışı, ileride Nesne Tabanlı Programlama'da (OOP) çok önemli konuların daha kolay kavranmasını sağlayacaktır. Özellikle self anahtar kelimesi gibi konuları anlamak için bu referans mantığı kritik öneme sahiptir.

- self anahtar kelimesi: Bir sınıf içindeki metotların, o metodu çağıran nesnenin kendisine referans vermesini sağlar. Bu sayede nesnenin kendi özelliklerine ve metotlarına erişilebilir.
- Nesne Referansları: Fonksiyonlara veya metotlara nesneleri argüman olarak geçirdiğimizde, aslında nesnenin kendisi değil, ona ait bir referans (işaretçi) geçmiş oluruz. Bu, bellek yönetimini daha verimli hale getirir.

PEP 8 İsimlendirme Kuralları: Neden Önemli?

PEP 8 (Python Enhancement Proposal 8), Python kodunun nasıl biçimlendirileceğine dair resmi bir stil rehberidir. Bu kurallar, kodun okunabilirliğini artırmak, tutarlılık sağlamak ve Python topluluğu içinde ortak bir yazım standardı oluşturmak için hayati öneme sahiptir.

PEP 8'e uymak şunları sağlar:

- Okunabilirlik: Aynı standartlarda yazılmış kod, farklı geliştiriciler tarafından bile kolayca anlaşılır ve sürdürülür.
- Tutarlılık: Tüm projelerde benzer isimlendirme ve biçimlendirme kullanılması, kod tabanının bütünlüğünü artırır.
- Profesyonellik: Endüstri standartlarına uygun kod yazmak, daha profesyonel bir geliştirici imajı çizer.



PEP 8 Değişken İsimlendirme Kuralları

Temel isimlendirme kuralları:

- Küçük Harf ve Alt Çizgi (snake_case): Değişken ve fonksiyon isimleri genellikle küçük harflerle yazılır ve kelimeler arasına alt çizgi (_) konur.
 - Doğru: kullanıcı_adi, toplam_fiyat, öğrenci_sayisi
 - Yanlış: KullaniciAdi, toplamfiyat, ÖğrenciSayisi
- Anlamlı İsimler: Değişken isimleri, ne tuttuklarını açıkça belirtmelidir. Tek harfli veya anlamsız isimlerden kaçınılmalıdır.
 - Doğru: müşteri_id, ürün_miktari
 - Yanlış: x, a, tmp
- Sayı ile Başlama Yasağı: Değişken isimleri bir sayı ile başlayamaz.
 - Doğru: sayı1, yeni_kullanıcı
 - Yanlış: 1sayı, 2_hafta
- Özel Karakterlerden Kaçınma: Değişken isimlerinde sadece harf, sayı ve alt çizgi kullanılmalıdır.



Dinamik Tiplendirme: Python'ın Ayırıcı Özelliği

Python, dinamik tiplendirme kullanan bir dildir. Bu, bir değişkenin veri tipinin, ona atanan değere göre otomatik olarak belirlenmesi anlamına gelir. Programcı, değişkeni tanımlarken tipini açıkça belirtmek zorunda değildir.

```
sayi = 10      # 'sayi' int tipinde olur
```

```
isim = "Ahmet"  # 'isim' str tipinde olur
```

```
pi = 3.14      # 'pi' float tipinde olur
```

```
sayi = "Merhaba"  # 'sayi' değişkeninin tipi str olarak değişir
```

Bu özellik, C++ veya Java gibi statik tipli dillerden gelen öğrenciler için önemli bir ayrım noktasıdır ve üzerinde durulmalıdır.



Dinamik Tiplendirme vs. Statik Tiplendirme

Statik Tipli Diller (C++, Java):

- Değişken tipi tanımlama anında veya derleme zamanında belirtilir ve sonradan değiştirilemez.
 - `int sayi = 10;`
 - `String isim = "Ayşe";`
- Derleme zamanında tip hataları yakalanır.
- Daha sıkı tip kontrolü ve genellikle daha yüksek performans sunar.

Dinamik Tipli Diller (Python, JavaScript):

- Değişken tipi çalışma zamanında (runtime) atanan değere göre belirlenir.
- Bir değişkenin tipi çalışma zamanında değişebilir.
- Daha hızlı geliştirme ve esneklik sağlar.
- Tip hataları genellikle çalışma zamanında ortaya çıkar.

Dinamik Tiplendirmenin Avantajları ve Dezavantajları

Avantajları:

- Esneklik: Bir değişkenin değerine ve tipine kolayca değiştirebilme yeteneği, hızlı prototipleme ve esnek kod yazımına olanak tanır.
- Hızlı Geliştirme: Tip bildirimine gerek olmaması, kodun daha kısa ve hızlı yazılmasını sağlar.
- Daha Az Kod: Aynı işi yapmak için genellikle daha az satır kod yazılır.

Dezavantajları:

- Çalışma Zamanı Hataları: Tip hataları derleme zamanında değil, program çalışırken ortaya çıkabilir, bu da hata ayıklamayı zorlaştırabilir.
- Performans: Tip kontrolü ve çözünürlüğü çalışma zamanında yapıldığı için, bazı durumlarda statik tipli dillere göre daha yavaş olabilir.
- Karmaşıklık Yönetimi: Büyük projelerde veya birden fazla geliştiriciyle çalışırken, tip belirsizliği kodu anlamayı ve sürdürmeyi zorlaştırabilir.

Temel Veri Tipleri: Giriş

Veri tipleri, programlama dillerinde verilerin ne tür bir bilgi taşıdığını ve o veri üzerinde ne tür işlemler yapılabileceğini belirtir. Python'da dört temel veri tipi bulunmaktadır ve bunları doğru bir şekilde kullanabilmek programlamanın temelini oluşturur.

Python'daki temel veri tipleri:

1. Integer (int): Tam sayılar.
2. Float (float): Ondalıklı sayılar.
3. String (str): Karakter dizileri.
4. Boolean (bool): Mantıksal değerler (Doğru/Yanlış).

Bu veri tipleri, programlarımızda sayısal hesaplamalar, metin işleme ve mantıksal kontroller yapmak için kullanılır.



Integer (Tam Sayılar)

Integer, matematikteki tam sayılar gibi, ondalık kısmı olmayan pozitif veya negatif sayıları temsil eder. Python'da integer'lar sınırsız hassasiyete sahiptir, yani çok büyük sayıları bile tutabilirler.

Özellikleri:

- Ondalık basamak içermezler.
- Negatif, sıfır veya pozitif olabilirler.
- Matematiksel işlemler (toplama, çıkarma, çarpma, bölme vb.) için kullanılırlar.

Örnekler:

```
yas = 25          # Pozitif tam sayı [4]
```

```
yıl = 2023
```

```
negatif_sayi = -10
```

```
sifir = 0
```

```
büyük_sayi = 12345678901234567890
```

`type()` fonksiyonunu kullanarak bir değişkenin tipini kontrol edebiliriz.

Float (Ondalıkli Sayılar)

Float, ondalık kısmı olan sayıları temsil eder, yani rasyonel ve irrasyonel sayıları ifade etmek için kullanılır. Bilimsel hesaplamalarda, finansal uygulamalarda ve ölçüm değerlerinde sıklıkla karşımıza çıkar.

- Ondalık basamak içerirler.
- Pozitif veya negatif olabilirler.
- Bilimsel notasyon (e veya E ile) da kullanılabilir (örneğin $2.5e-3 = 0.0025$).

Örnekler:

boy = 1.75 # Ondalıkli sayı [4]

pi = 3.14159

para_miktari = 19.99

sıcaklık = -5.6

bilimsel = 6.022e23 # $6.022 * 10^{23}$

Float değerler üzerinde yapılan hesaplamalarda, kayan nokta hassasiyetinden kaynaklanan küçük hatalar oluşabileceğini unutmamak önemlidir.

String (Karakter Dizileri)

String, metin verilerini temsil etmek için kullanılan bir karakter dizisidir. Python'da string'ler tek tırnak (' '), çift tırnak (" ") veya üçlü tırnak (' ' ' ' veya " " " ") içine alınarak tanımlanır.

- Metin verilerini depolamak için kullanılırlar.
- İçindeki karakterlere indeksleme ile erişilebilir.
- Birçok hazır metin işleme metoduna sahiptirler (büyük/küçük harf dönüştürme, birleştirme vb.).

Örnekler:

```
isim = "Ali"           # Çift tırnak ile string tanımlama [4]
mesaj = 'Merhaba Dünya!' # Tek tırnak ile string tanımlama
uzun_metin = """Bu,
çok satırlı bir metindir.""" # Üçlü tırnak ile çok satırlı string
boş_string = ""
```

String'ler değiştirilemez (immutable) veri tipleridir, yani bir string oluşturulduktan sonra içeriği değiştirilemez; ancak yeni bir string oluşturulabilir.

Boolean (Mantıksal Değerler)

Boolean, sadece iki olası değerden birini alabilen bir veri tipidir: True (Doğru) veya False (Yanlış). Bu değerler genellikle koşullu ifadelerde, mantıksal karşılaştırmalarda ve program akışını kontrol etmek için kullanılır.

Özellikleri:

- Sadece True veya False değerlerini alabilirler.
- Python'da True ve False özel anahtar kelimelerdir ve ilk harfleri büyük yazılmalıdır.
- Karşılaştırma operatörlerinin (==, >, <, >=, <=, !=) sonuçları genellikle boolean değerlerdir.

Örnekler:

```
ogrenci_mi = True      # Boolean değeri [4]
```

```
ders_bitti_mi = False
```

```
sonuc = (5 > 3)        # sonuc True olur
```

```
esit_mi = (10 == 20)   # esit_mi False olur
```

Boolean değerler, programlamada karar verme mekanizmalarının temelini oluşturur.

type() Fonksiyonu: Dinamik Tip Kontrolü

Python'da bir değişkenin veri tipini çalışma zamanında (dinamik olarak) kontrol etmek için `type()` fonksiyonunu kullanırız. Bu fonksiyon, bir değişkenin veya bir değer ait olduğu sınıfın tipini döndürür. Bu, dinamik tiplendirmenin bir sonucudur ve özellikle kodun farklı yerlerinde bir değişkenin ne tür bir veri tuttuğunu anlamak için faydalıdır.

Kullanımı: `type(değişken_adı)` veya `type(değer)`



type() Fonksiyonu: Uygulama Örneği

Farklı veri tiplerinde değişkenler tanımlanır ve her birinin tipi `type()` fonksiyonu ile ekrana yazdırılır.

Kod Uygulaması:

```
# Farklı veri tiplerinde değişkenler tanımlayalım [4]
```

```
yas = 25          # Integer (tam sayı)
```

```
boy = 1.75        # Float (ondalıklı sayı)
```

```
isim = "Ali"      # String (karakter dizisi)
```

```
ogrenci_mi = True # Boolean (mantıksal değer)
```

```
# Her bir değişkenin tipini type() fonksiyonu ile ekrana yazdıralım [4]
```

```
print(f"yas değişkeninin tipi: {type(yas)}")
```

```
print(f"boy değişkeninin tipi: {type(boy)}")
```

```
print(f"isim değişkeninin tipi: {type(isim)}")
```

```
print(f"ogrenci_mi değişkeninin tipi: {type(ogrenci_mi)}")
```

type() Fonksiyonu: Uygulama Örneği

Çıktı:

yas değişkeninin tipi: <class 'int'>

boy değişkeninin tipi: <class 'float'>

isim değişkeninin tipi: <class 'str'>

ogrenci_mi değişkeninin tipi: <class 'bool'>



input() Fonksiyonu: Kullanıcıdan Veri Alma

input() fonksiyonu, bir programın kullanıcıdan klavye aracılığıyla veri almasını sağlayan temel bir fonksiyondur. Kullanıcıdan alınan tüm veriler varsayılan olarak string (karakter dizisi) tipindedir. Bu önemli bir detaydır, çünkü sayısal işlemler yapmadan önce bu string'leri sayısal tiplere dönüştürmek gerekebilir.

Kullanımı: `degisken_adi = input("Kullanıcıya gösterilecek mesaj: ")`



print() Fonksiyonu: Ekrana Yazdırma

`print()` fonksiyonu, Python'da verileri veya mesajları konsola (ekrana) yazdırmak için kullanılan temel fonksiyondur. Debugging (hata ayıklama) sırasında veya kullanıcıya bilgi vermek için yaygın olarak kullanılır.

Kullanımı:

- `print("Mesaj")`
- `print(değişken_adi)`
- `print("Mesaj:", değişken_adi)` (Birden fazla argüman virgülle ayrılabilir)
- `print(f"F-string ile mesaj: {değişken_adi}")` (Formatlanmış string literal - f-string)

Ders İçi Kodlama Örneği: İsim Alma ve Karşılama

Kullanıcıdan ismini alıp "Merhaba, [Kullanıcı Adı]!" şeklinde ekrana yazdıran basit bir program, input() ve print() fonksiyonlarının temel kullanımını gösterir.

Kod Uygulaması:

```
# Kullanıcıdan ismini alalım [4]
```

```
kullanici_adi = input("Lütfen isminizi girin: ")
```

```
# Alınan ismi kullanarak bir karşılama mesajı oluşturalım ve ekrana yazdıralım [4]
```

```
print("Merhaba,", kullanici_adi + "!")
```

```
# Alternatif olarak f-string kullanarak:
```

```
# print(f"Merhaba, {kullanici_adi}!")
```

Veri Tipleri Arasında Dönüşüm: Neden Gerekli?

`input()` fonksiyonundan dönen değerlerin varsayılan olarak string olduğu gerçeği, matematiksel işlemler yapmadan önce bu string'leri `int()` veya `float()` gibi sayısal tiplere dönüştürme ihtiyacını doğurur. Eğer dönüşüm yapılmazsa, Python string'leri metin olarak ele alacak ve beklenmedik sonuçlar verecektir (örneğin `"5" + "3" = "53"` yerine 8 değil).

Temel dönüştürme fonksiyonları:

- `int()`: Bir değeri tam sayıya dönüştürür.
- `float()`: Bir değeri ondalıklı sayıya dönüştürür.
- `str()`: Bir değeri karakter dizisine dönüştürür.



int(), float(), str() Fonksiyonları

int(değer):

- Bir string'i veya float'u tam sayıya dönüştürür.
- String'in sadece tam sayı içermesi gerekir. Float'lar virgülden sonraki kısmını atar.

```
sayi_str = "123"
```

```
sayi_int = int(sayi_str) # 123 (int)
```

```
ondalik_float = 5.7
```

```
ondalik_int = int(ondalik_float) # 5 (int)
```

int(), float(), str() Fonksiyonları

float(değer):

- Bir string'i veya int'i ondalıklı sayıya dönüştürür.
- String'in sayısal bir formatta olması gerekir.

```
fiyat_str = "49.99"
```

```
fiyat_float = float(fiyat_str) # 49.99 (float)
```

```
tam_sayi_int = 10
```

```
tam_sayi_float = float(tam_sayi_int) # 10.0 (float)
```

int(), float(), str() Fonksiyonları

str(değer):

- Herhangi bir veri tipini (int, float, bool vb.) karakter dizisine dönüştürür.

```
yas_int = 30
```

```
yas_str = str(yas_int) # "30" (str)
```

```
aktif_bool = True
```

```
aktif_str = str(aktif_bool) # "True" (str)
```

Ders İçi Kodlama Örneği: Sayı Toplama (Dönüşümle) – Problem

Kullanıcıdan iki sayı alıp toplamaya çalışan bir kod yazacağız. Ancak `input()` fonksiyonundan dönen değerlerin varsayılan olarak string olduğunu ve bu nedenle matematiksel işlem yapmadan önce `int()` veya `float()` ile dönüşüm yapılması gerektiğini uygulamalı olarak göstereceğiz.

Problem: Kullanıcıdan alınan "5" ve "3" string değerlerini toplayıp 8 sonucunu veren bir program yazmak. Eğer dönüşüm yapmazsak, sonuç "53" string'i olacaktır.



Ders İçi Kodlama Örneği: Sayı Toplama (Dönüşümle) – Kod

```
# Kullanıcıdan birinci sayıyı alalım
sayi1_str = input("Birinci sayıyı girin: ")

# Kullanıcıdan ikinci sayıyı alalım
sayi2_str = input("İkinci sayıyı girin: ")

# Eğer dönüşüm yapmazsak ne olur? (Hatalı örnek)
# toplam_hatali = sayi1_str + sayi2_str

# print(f"Hatalı toplam (string birleştirme):
# {toplam_hatali}")

# Çıktı: Hatalı toplam (string birleştirme): 53 (eğer
# kullanıcı 5 ve 3 girerse)

# Doğru işlem için string'leri int'e dönüştürelim [4]
sayi1 = int(sayi1_str)
sayi2 = int(sayi2_str)

# Sayıları toplayalım
toplam = sayi1 + sayi2

# Sonucu ekrana yazdıralım
print(f"Sayıların toplamı: {toplam}")

# Eğer kullanıcı ondalıklı sayı girecekse float()
# kullanmalıyız.
```

Şimdiye kadar öğrendiğimiz temel bilgileri ve fonksiyonları kullanarak pratik uygulamalar yapacağız. Bu uygulamalar, teorik bilginizi pekiştirmenize ve gerçek dünya problemlerine Python ile çözüm üretme yeteneğinizi geliştirmenize yardımcı olacaktır.

Uygulamalar:

1. Vücut Kitle Endeksi (VKİ) Hesaplayıcı.
2. Basit Fatura Hesaplama.

Bu alıştırmalar, `input()`, `print()`, `int()`, `float()` gibi fonksiyonları ve değişken tanımlamayı etkin bir şekilde kullanmanızı gerektirecektir.

VKİ Hesaplayıcı: Problem Tanımı

Vücut Kitle Endeksi (VKİ) Hesaplayıcı: Kullanıcıdan boy (metre cinsinden) ve kilo (kilogram cinsinden) bilgilerini alan, bu değerleri float tipine dönüştüren ve $VKİ = \text{kilo} / (\text{boy} * \text{boy})$ formülünü kullanarak sonucu ekrana yazdıran bir program geliştireceğiz.

Adımlar:

1. Kullanıcıdan boy bilgisini `input()` ile alın.
2. Kullanıcıdan kilo bilgisini `input()` ile alın.
3. Alınan string değerleri `float()` fonksiyonu ile ondalıklı sayıya dönüştürün.
4. Verilen formülü kullanarak VKİ'yi hesaplayın.
5. Hesaplanan VKİ değerini `print()` ile ekrana yazdırın.

```
# 1. Kullanıcıdan boy (metre) bilgisini alalım
# input() varsayılan olarak string döndürür, bu yüzden dönüştürmemiz
# gerekecek
boy_str = input("Lütfen boyunuzu metre cinsinden girin (örn: 1.75): ")

# 2. Kullanıcıdan kilo (kilogram) bilgisini alalım
kilo_str = input("Lütfen kilonuzu kilogram cinsinden girin (örn: 70.5): ")

# Bu noktada 'boy_str' ve 'kilo_str' değişkenleri string tipli değerler
# içerir.
# Matematiksel işlem yapabilmek için bu string'leri float'a
# dönüştürmeliyiz [5].
```

VKİ Hesaplayıcı: Kod (Sayfa 2/2)

```
# 3. String değerleri float tipine dönüştürelim [5]
```

```
boy = float(boy_str)
```

```
kilo = float(kilo_str)
```

```
# 4. VKİ formülünü kullanarak sonucu hesaplayalım [5]
```

```
# VKİ = kilo / (boy * boy)
```

```
vki = kilo / (boy * boy)
```

```
# 5. Hesaplanan VKİ'yi ekrana yazdıralım [5]
```

```
print(f"\nVücut Kitle Endeksiniz (VKİ): {vki:.2f}") # .2f ile iki ondalık basamak gösteriyoruz
```

```
# Not: Programın hata toleransı eklenerek (örneğin sayısal olmayan girişleri kontrol ederek)
```

```
# daha sağlam hale getirilebileceğini unutmayın. Bu, ileriki derslerde işlenebilir.
```

Basit Fatura Hesaplama: Problem Tanımı

Basit Fatura Hesaplama: Kullanıcıdan bir ürünün KDV'siz fiyatını ve KDV oranını (%) alıp, ürünün KDV'li toplam fiyatını hesaplayıp ekrana yazdıran bir program geliştireceğiz.

Adımlar:

1. Kullanıcıdan KDV'siz fiyatı `input()` ile alın.
2. Kullanıcıdan KDV oranını (yüzde olarak, örn: 18) `input()` ile alın.
3. Alınan string değerleri `float()` tipine dönüştürün.
4. KDV miktarını ve KDV'li toplam fiyatı hesaplayın.
 - $\text{KDV Miktarı} = \text{KDV'siz Fiyat} * (\text{KDV Oranı} / 100)$
 - $\text{KDV'li Toplam Fiyat} = \text{KDV'siz Fiyat} + \text{KDV Miktarı}$
5. Sonucu `print()` ile ekrana yazdırın.

Çalışma Soruları

Hafta 1 – Python Nesne Tabanlı Programlamaya Giriş

Çalışma Soruları

1. Dersin ilk haftasının temel hedefleri nelerdir?
2. Python'un backend geliştirme ve veri bilimi gibi alanlarda popüler olmasının temel nedenleri nelerdir?
3. Ders notlarına göre Python'da bir "değişken" kavramı nasıl açıklanmaktadır ve bu anlayış neden önemlidir?
4. Dinamik tiplendirme (dynamic typing) nedir ve statik tipli dillerden temel farkı nedir?
5. input() fonksiyonu kullanıcıdan veri aldığı anda bu verinin varsayılan veri tipi ne olur?
6. Kullanıcıdan alınan sayısal verilerle matematiksel işlem yapmadan önce neden tip dönüşümü yapılması zorunludur?
7. type() fonksiyonunun temel işlevi nedir?
8. Ders kapsamında tanıtılan dört temel veri tipi hangileridir?
9. Vücut Kitle Endeksi (VKİ) hesaplayıcı laboratuvar uygulamasının amacı ve işleyişini kısaca açıklayınız.
10. Dersin başlangıcında eğitmenin öğrencilere aktarması gereken idari ve yapısal bilgiler nelerdir?

Deneme Soruları

Hafta 1 – Python Nesne Tabanlı Programlamaya Giriş

Deneme Soruları

1. Ders notlarında vurgulanan "değişkenlerin bellekteki nesnelere birer referans olduğu" fikrinin, Nesne Tabanlı Programlama'nın ilerleyen konuları (örneğin, nesne referansları ve self anahtar kelimesi) için neden kritik bir temel oluşturduğunu detaylı bir şekilde tartışınız.
2. Bir kullanıcının girdiği iki sayının toplamını hesaplayan bir Python programının tüm adımlarını ayrıntılı olarak açıklayınız. Bu süreçte `input()`, `int()` veya `float()`, matematiksel operatörler ve `print()` fonksiyonlarının rollerini ve birbirleriyle olan ilişkisini analiz ediniz.
3. Python'un dinamik tiplendirme özelliğinin, C++ veya Java gibi statik tipli dillerden gelen bir öğrenci için ne gibi avantajlar ve zorluklar sunabileceğini ders notları ışığında değerlendiriniz.
4. Hafta 1'deki hedefler ve laboratuvar uygulamaları (VKİ ve Fatura Hesaplama) göz önüne alındığında, Python'un basit sözdizimi ve yerleşik fonksiyonlarının pratik problem çözümünü nasıl kolaylaştırdığını örneklerle açıklayınız.
5. Ders notlarındaki "Teorik Konular ve Öğitmen Notları" bölümünü temel alarak, bir öğretmenin ilk ders gününü nasıl yapılandırması gerektiğini anlatan bir ders akış planı oluşturunuz. Bu plan, Python'un popülerliğinin nedenlerinden değişken kavramının derinliğine kadar tüm teorik konuları nasıl ve ne sırayla sunmalıdır?

ÖZET



Süleyman **EZDEMİR**

suleyman.ezdemir@giresun.edu.tr

